Universidade do Minho

Ano Letivo: 2025/26

Databases

Introduction to SQL



Summary

1 Logical Model Review

2 Introduction to SQL

3 Data Description - DDL

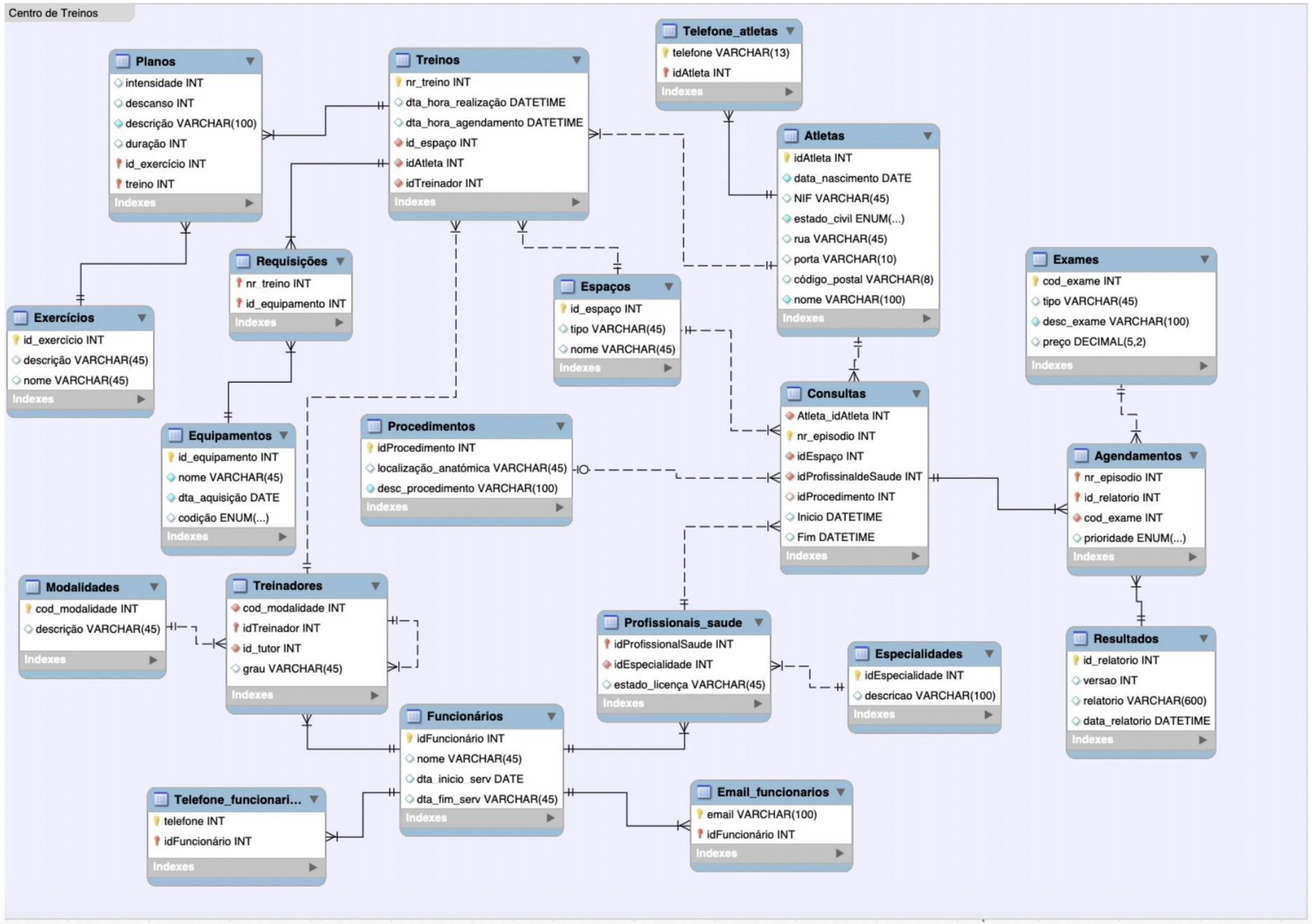
4 Updating and Manipulating SQL Data - DML

Bibliography:

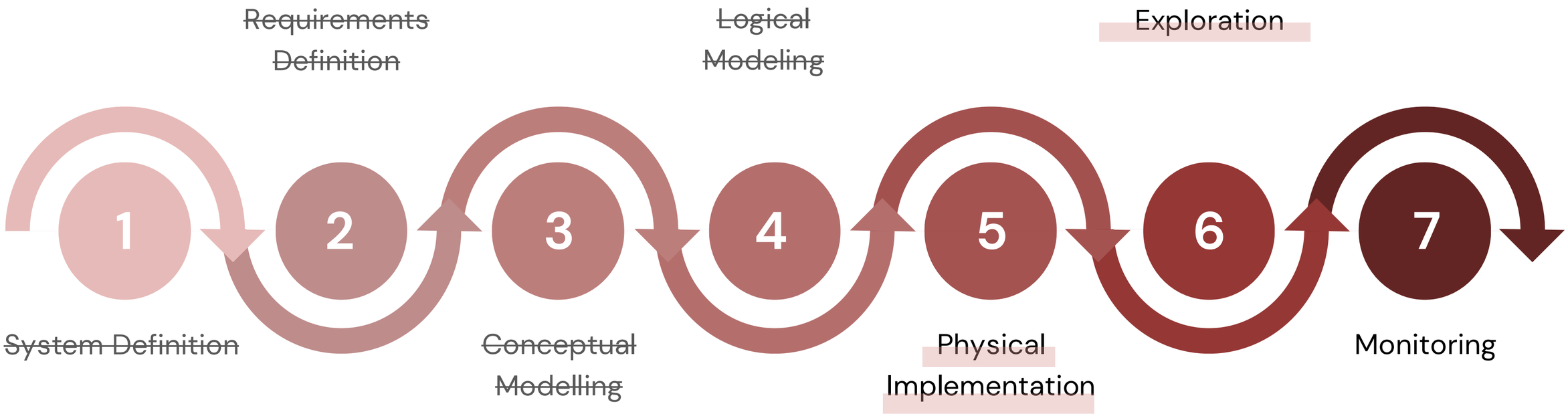
- Connolly, T., Begg, C., Database Systems, A Practical Approach to Design, Implementation, and Management , Addison-Wesley, 4a Edição, 2004. **(Chapter 18)**

- Belo, O., "Bases de Dados Relacionais: Implementação com MySQL", FCA – Editora de Informática, 376p, Set 2021. ISBN: 978-972-722-921-5. **(Chapter 2)**

Modelo Relacional
Caso de Estudo



Database System Lifecycle



Database Languages

Introduction

Databases require mechanisms that allow users and applications to interact with data in a structured and efficient way.
These mechanisms are provided through database languages, which should enable a user to:

- create the database and the structures of relations;
- perform basic data management tasks, such as inserting, updating, and deleting data in relations;
- execute simple and complex queries.

A database language should perform these tasks with minimal user effort, and its structure and syntax should be easy to learn.

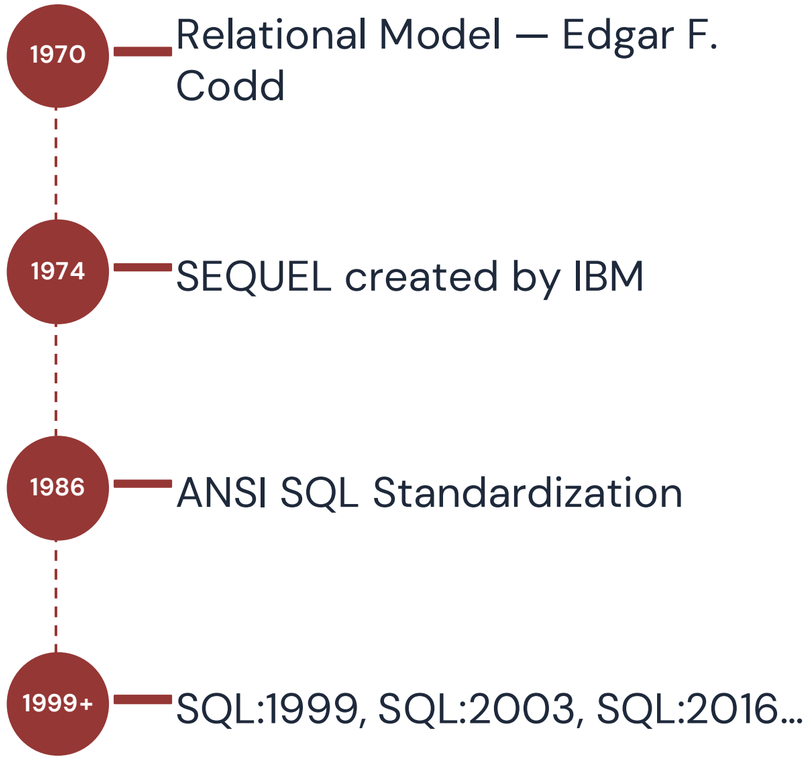
The SQL Language

What is SQL?

SQL (Structured Query Language) is the standard language for interacting with Relational Database Management Systems (RDBMS).

It was developed by IBM in the 1970s and standardized by ANSI/ISO.

It is declarative: it specifies WHAT we intend to obtain, not HOW to obtain it.



The SQL Language

DBMSs

SQL is supported by most Database Management Systems (DBMS), such as:







Although there are slight variations between systems (dialects), the basis of the language remains common.

SQL is widely used in:

- Web applications
- Enterprise Systems
- Mobile apps
- Data analysis systems

The SQL Language

SGBDs

MySQL is the RDBMS that will be used throughout this course.

With MySQL you can:

- Create and structure databases
- Store and manage information
- Run SQL queries
- Ensure data integrity and security



MySQL is an open-source tool, widely used in both academia and industry.

It is currently maintained by Oracle Corporation.

The SQL Language

Operations

DDL — Data Definition Language

Defines the structure of the database

`CREATE`
`ALTER`
`DROP`

DML — Data Manipulation Language

Manipulates the data

`INSERT`
`UPDATE`
`DELETE`

DQL — Data Query Language

Consult the data:

`SELECT`

TCL — Data Control Language

Controls access permissions

`GRANT`
`REVOKE`

The SQL Language

DDL

Data Description

The Data Definition Language (DDL) is used to define and modify the structure of the database.

It allows you to create, change and delete objects from the database, such as:

- Tabelas
- Indexes
- Schemas
- Views

DDL commands act directly on the structure and not on the data itself.
Main commands:

- `CREATE`
- `ALTER`
- `DROP`
- `TRUNCATE`

The SQL Language

DDL

Create Database

CREATE DATABASE

The CREATE DATABASE command is used to create a new database.

Structure

```
CREATE {DATABASE | SCHEMA} [IF NOT EXISTS] db_name
[create_option] ...
create_option: [DEFAULT] {
CHARACTER SET [=] charset_name
| COLLATE [=] collation_name
| ENCRYPTION [=] {'Y' | 'N'}}
```

The SQL Language

DDL

Create Database

CREATE DATABASE

The CREATE DATABASE command is used to create a new database.

Example:

This part is optional

```
CREATE DATABASE IF NOT EXISTS escola;
```

In this case, a database is created with the name 'escola'

To use the database you created:

```
USE escola;
```

From this moment on, all operations will be carried out within this database.

`DROP DATABASE` – Removal of a database from a system.

The SQL Language

DDL

Create Tables

Tables

In relational database systems, information is stored in tables (also called relations).

The main operations for defining data on tables are:

- **CREATE TABLE** — allows you to create the structure of a new table in the database
- **ALTER TABLE** — allows you to modify the structure of an existing table
- **DROP TABLE** — allows you to remove a table from the database

The SQL Language

DDL

Create Tables

CREATE TABLE

```
CREATE TABLE [IF NOT EXISTS ] <nome_tabela> (
<nome_coluna> <tipo_coluna[tamanho]> [NOT NULL | NULL] [DEFAULT <value>]
[AUTO_INCREMENT][UNIQUE],
…,
PRIMARY KEY (<nome_coluna_PK>,…)
[CONSTRAINT <constraint_name>] UNIQUE {KEY | INDEX} (<nome_coluna>,…)
[CONSTRAINT <constraint_name>] FOREIGN KEY (<nome_coluna_FK>) REFERENCES <nome_tabela_FK>
(<nome_coluna_FK>)
[ON UPDATE <referential_integrity_constraint>] [ON DELETE
<referential_integrity_constraint>]
) [ENGINE=<storage_engine>];
```

If ENGINE is not declared, MySQL uses InnoDB by default.
referential_integrity_constraint = {NO ACTION | RESTRICT | CASCADE | SET NULL | SET DEFAULT } If you do not specify the ON clause DELETE e ON UPDATE, o MySQL will use the standard definition.

The SQL Language

DDL

Data Description

Data Types

In SQL, each column of a table must have a data type, which defines the type of values it can store (domain). The main types of data are:

Numerical
INT — intigers
DECIMAL(p,s) — numbers with decimal part
FLOAT — real numbers(approximate)

Text
CHAR(n) — Fixed-size text
VARCHAR(n) — Variable size text

Booleano
BOOLEAN — true/false values(in MySQL: 0 ou 1)

Date and Time
DATE — date (YYYY-MM-DD)
TIME — hour
DATETIME — date and hour

Outros
BLOB — binary data (ex: Images, Files)
TEXT — Long texts

The SQL Language

DDL

Data Description

Constraints

Constraints are rules applied to the columns of a table to ensure the integrity and consistency of the data. They allow you to control what type of values can be entered into the database.

PRIMARY KEY
- Uniquely identifies each record
- Cannot contain NULL values
- Cannot have duplicate values

NOT NULL
- Ensures that the column cannot have empty values

UNIQUE
- Prevents duplicate values in a column

UNSIGNED
- Prevents negative values

FOREIGN KEY
- Connect tables
- Ensures referential integrity

CHECK
- Defines a condition that values must respect

DEFAULT
- Sets a default value for the column

The SQL Language

DDL

Create Tables

CREATE TABLE

The CREATE TABLE command allows you to create a new table within the database.

```
CREATE TABLE Student (
  id    INT   AUTO_INCREMENT  PRIMARY KEY,
  name       VARCHAR(100) NOT NULL,
  email      VARCHAR(150) UNIQUE,
  birth_date DATE
);
```

Example 1:

- The students table is created
- Attributes and data types are defined
- The id column is set as the primary key
- The id value is automatically incremented with each new record
- Name is mandatory
- And the email can’t be repeated

The SQL Language

DDL

Create Tables

CREATE TABLE

```
CREATE TABLE Course (
  id     INT    PRIMARY KEY,
  lescription      VARCHAR(150) NOT NULL,
  limit      INT CHECK (limite >= 0),
  state VARCHAR(20) DEFAULT ‘ativo’

);
```

Example 2:

- The course table is created
- Attributes and data types are defined
- The id column is set as the primary key
- The limit has a condition
- The state has a default value

The SQL Language

DDL

Create Tables

CREATE TABLE

```
CREATE TABLE Enrollment (
  student_id    INT,
  course_id     INT,
  PRIMARY KEY (student_id, course_id),
  FOREIGN KEY (student_id) REFERENCES student(id),
  FOREIGN KEY (course_id) REFERENCES course(id)

);
```

Example 3:

- The enrollment table is created
- Attributes and data types are defined
- The primary key is composed
- 2 foreign keys are defined (Referential Integrity)

The SQL Language

DDL

Updating Tables

After creating a table, you may need to modify its structure. These changes can occur for several reasons:

- New System Requirements
- Fixing modeling errors
- Evolution of the database over time

To make these changes, SQL provides the command:

```
ALTER TABLE
```

This command allows you to:

- Add new columns
- Remove existing columns
- Change data types
- Add or remove restrictions

The SQL Language

DDL

Updating Tables

ALTER TABLE

ALTER TABLE <nome_tabela_antigo> RENAME TO <nome_tabela_novo>; → altera o nome da tabela.

ALTER TABLE <nome_tabela> ADD <nome_campo> <domínio_campo>; → cria um novo atributo na tabela com o nome e domínio especificados. Todos os tuplos existentes ficam com NULL no novo atributo.

ALTER TABLE <nome_tabela> DROP <nome_campo>; → apaga o atributo com o nome especificado da tabela.

ALTER TABLE <nome_tabela> MODIFY <nome_campo> <domínio_campo>; → modifica o atributo com o nome especificado da
tabela.

ALTER TABLE <nome_tabela> ALTER < nome_campo> SET DEFAULT <value>; → modifica uma coluna de uma tabela para lhe atribuir valores padrão.

ALTER TABLE <nome_tabela> ALTER < nome_campo> DROP DEFAULT; → remove os valores padrão de uma coluna de uma tabela.

ALTER TABLE <nome_tabela> ADD CONSTRAINT <nome_constraint> UNIQUE KEY(column_1,column_2,...); → modifica uma tabela para lhe atribuir uma indexação única .

The SQL Language

DDL

Updating Tables

ALTER TABLE

Examples:

• Added a new phone attribute to the students table:

```
ALTER TABLE students
ADD NIF INT;
```

• Added new attributes with position indication:

```
ALTER TABLE students
ADD phone VARCHAR(13) AFTER nome,
ADD type VARCHAR(50) FIRST;
```

• Removing an attribute from a table:

```
ALTER TABLE students
DROP COLUMN type;
```

• Modifying the definition of an attribute:

```
ALTER TABLE students
MODIFY name VARCHAR(150) NOT NULL;
```

• Modification of several attributes of a table:

```
ALTER TABLE students
MODIFY name VARCHAR(150) NOT NULL,
MODIFY age INT UNSIGNED;
```

• Changing the name of an attribute:

```
ALTER TABLE students
CHANGE name complete_name VARCHAR(150);
```

The SQL Language

DDL

Removing Tables

Removing a table from a database is performed using the instruction:
DROP TABLE

When a table is removed, the following are also deleted:
- All records (data)
- Indexes and triggers
- Constraints
- Associated privileges
- The table definition from the database

⚠ **Important:**
If a table is referenced by a **foreign key**, it cannot be removed directly.
In that case, it is necessary to:
- Remove the foreign key constraint first
- Or eliminate the existing dependencies

Key idea:
DROP TABLE completely removes the table and everything associated with it.

The SQL Language

DDL

Removing Tables

DROP TABLE

Examples:

• Removing a table:

```
DROP TABLE alunos;
```

• Removing multiple tables:

```
DROP TABLE alunos, cursos;
```

• Removal only if the table exists:

```
DROP TABLE IF EXISTS alunos;
```

The SQL Language

DML

Data Manipulation

The Data Manipulation Language (DML) corresponds to the set of SQL instructions used to manipulate the data stored in the tables.

These instructions allow you to:

- Insert new records
- Modify existing data
- Remove data from the database

DML operations act directly on the contents of the tables, without changing their structure.

Key Instructions:

- `INSERT` — data entry
- `UPDATE` — data modification
- `DELETE` — data Removal

The SQL Language

DML

Data Entry

Entering data into a table is performed by instructing:

```
INSERT INTO
```

Structure:

```
INSERT [LOW_PRIORITY | DELAYED | HIGH_PRIORITY] [IGNORE]
[INTO] tbl_name
[PARTITION (partition_name [, partition_name] ...)]
[(col_name [, col_name] ...)]
{ {VALUES | VALUE} (value_list) [, (value_list)] ... }
[AS row_alias[(col_alias [, col_alias] ...)]]
[ON DUPLICATE KEY UPDATE assignment_list]
(…)
```

The SQL Language

DML

Data Entry

INSERT INTO

Examples:

- Inserting a new record:

```
INSERT INTO Curso (id, descrição, limite, estado)
VALUES (1, 'Programação', 30, 'ativo’);
```

Or

```
INSERT INTO Curso VALUES (1, 'Programação', 30, 'ativo’);
```

- Inserting multiple records:

```
INSERT INTO Curso (id, descrição, limite)
VALUES
(2, ‘Biologia’, 20),
(3, ‘Música’, 10);
```

In this case the state attribute is not specified → gets the value 'active' because it had been specified as default

The SQL Language

DML

Data Entry

INSERT INTO

Examples:

Inserting a record into a table, specifying only some of the attributes defined in the table schema:

```
INSERT INTO Aluno (nome, email)
VALUES ('Ana Silva', 'ana@email.com');
```

In this case:
The id attribute is automatically generated and incremented (AUTO_INCREMENT)
The data_nasc attribute is not specified → is left with a NULL value

The SQL Language

DML

Data Modification

The data change is carried out by instructing:

`UPDATE`

Structure:

```
UPDATE [LOW_PRIORITY] [IGNORE] table_reference
SET assignment_list
[WHERE where_condition]
[ORDER BY ...]
[LIMIT row_count]
value:
{expr | DEFAULT}
assignment:
col_name = value
assignment_list:
assignment [, assignment] ...
```

The SQL Language

DML

Data Modification

UPDATE

Examples

• Updating records (affects all rows in the table):

```
UPDATE cursos
SET estado = 'ativo',
```

• Updating multiple attributes with a condition (only affects rows that meet the condition):

```
UPDATE cursos
SET estado = 'inativo', limite = '0'
WHERE id IN (1,2);
```

⚠ Without the WHERE clause, all records in the table will be modified.

The SQL Language

DML

Data Removal

Data removal is performed through the instruction:

```
DELETE
```

Structure:

```
DELETE [LOW_PRIORITY] [QUICK] [IGNORE] FROM tbl_name [[AS] tbl_alias]
[PARTITION (partition_name [, partition_name] ...)]
[WHERE where_condition]
[ORDER BY ...]
[LIMIT row_count]
```

The SQL Language

DML

Data Removal

DELETE

Examples

- Removing a record:

```
DELETE FROM alunos
WHERE id = 1;
```

- Removal of multiple records:

```
DELETE FROM alunos
WHERE YEAR(data_nasc) < 2018;
```

- Removing all records from a table:

```
DELETE FROM alunos;
```

```
-- Disable Safe Mode
SET SQL_SAFE_UPDATES=0;
-- Enable Safe Mode
SET SQL_SAFE_UPDATES=1;
```

⚠ Without the WHERE clause, all records in the table will be removed.

Universidade do Minho

Ano Letivo: 2025/26

Databases

Introduction to SQL



António Abelha | abelha@di.uminho.pt